43008 Reinforcement Learning: Assessment 3

Train Schedule Optimisation under Disruptions

Learning to change track when a corridor is blocked

Student	ID
Daniel James Martirosov	24948933
Laurean Lim	25049863
Ziheng Deng	25595315

Team RailTech, Project 27. Spring 2026, submitted 25 October 2026.

Code deliverable: notebooks/train_schedule_optimisation.ipynb
Repository: MathgeniusTB2/RL-Assessment-3 (Flatland 4.3.0, Python 3.11)

Abstract

This report asks whether a learned policy can beat a schedule-following controller when a track becomes unavailable. Two Flatland scenarios are built from the October 2026 TfNSW GTFS feed: a four-track Central-Strathfield corridor with 52 scheduled movements and the Granville-Harris Park junction with 126. The contribution is as much methodological as empirical. A train-borne disruption layer fires only when a train reaches the chosen cell, so a policy that reroutes early can make the incident vanish: in our tests a rerouting checkpoint suppressed the incident and scored exactly as it did on a clean episode. The corridor therefore uses an exogenous blockage: a cell's rail transitions are removed for 400 steps at a seed-drawn time, so both arms face the same cells at the same planned steps. Under that regime, and across all three algorithms, only the warm-started PPO learns to change track: it improves the metric from -0.0662 to -0.0005, a paired gain of +0.0657 on 30 held-out seeds (30 of 30, $p < 1e-4$) with every train completing, and on a clean network it scores 0.0000 and turns in none of 546 clear-track decisions. A2C never learns the crossing (best checkpoint -0.0563) and DQN parks half the fleet. The junction remains a loss whose failure mode changed with the observation: the selected checkpoint now dispatches and jams (-0.2400, with 59 of 126 trains completing) instead of parking (-0.0251, 0 of 126), and it still finishes behind the scripted route follower (-0.1823, 126 of 126), because the metric charges a cancellation less than a jammed dispatch.

1 Introduction

A suburban rail corridor is a scheduling problem wrapped in a control problem. The timetable is fixed, the track layout offers few alternatives, and the failures that matter are the ones that remove a cell of track for longer than the headway behind it. Classical rescheduling solves this with a central optimiser that has the whole fleet in view. The question here is narrower and deliberately local: with one shared policy controlling every train from its own observation, can a learner discover that it should change track before a blockage, and can that behaviour be measured honestly?

Flatland is a natural testbed: a grid of rail cells, discrete movements, switches and crossovers, and a metric that charges lateness, missed stops and cancellations. We build two scenarios from real GTFS data rather than from synthetic track. The first is a condensed version of the Main Suburban corridor between Central and Strathfield, four adjacent through tracks with stations at Strathfield, Burwood, Redfern and Central. The second is the Main Western junction at Granville, where the Suburban pair and the Old Main South converge at grade; the one grade separation in the level is the Up Old Main South crossing the Y-link on a flyover, modelled as level-free cells.

Contributions. The report makes six claims that a reader can check against the shipped evidence.

- A reproducible corridor and a junction built from the October 2026 GTFS feed, with levels, fleet and timetable shipped as fixed inputs, and one notebook that runs both scenarios.
- The assessment metric used as the training reward, paid densely as a potential, r(t) = R(t) - R(t-1); the sum telescopes to the metric exactly, verified per agent and per term to machine precision, and training divides that potential by a constant 100 so the value targets stay O(1).
- A negative diagnostic result about disruption placement: in the retired band regime no sampled incident left a train able to reach its crossover (0 of 9, results/all_arms.md), and a train-borne failure can be suppressed entirely by the policy whose behaviour is being measured.
- An exogenous blockage regime that fixes both, plus a policy whose crossings are conditional: 81 of 101 crossover visits with a blocked cell ahead produced a turn, and 0 of 202 with a clear track did.

- A three-algorithm comparison on the same regime: only the warm-started PPO reroutes; A2C stays near the baseline without learning the crossing, and DQN parks half the fleet.
- An honest paired result on 30 held-out seeds for the corridor (+0.0657, no train lost) and a junction result that is reported as a loss, with its metric pathology identified.

2 Overview of the architecture and system

The system is a simulation-based reinforcement-learning pipeline rather than a deployed control system. It has three separable layers: the scenarios (level, timetable, fleet), the learning layer (observation, reward, algorithms), and the evaluation layer (paired tests on held-out seeds plus a decision audit). Keeping them separable is what allowed the disruption layer to be replaced without touching the policy or the metric.

2.1 Workflow

The workflow runs in seven stages, each with a defined input and output (Figure 1); the first two, service extraction and pre-processing, are described together below. The dashed arrow is deliberate: every change to the observation, reward or disruption regime was re-validated by re-running the same paired evaluation, so the report only contains numbers produced by the final configuration, with the two labelled junction rows of Table 5 as the one disclosed exception.

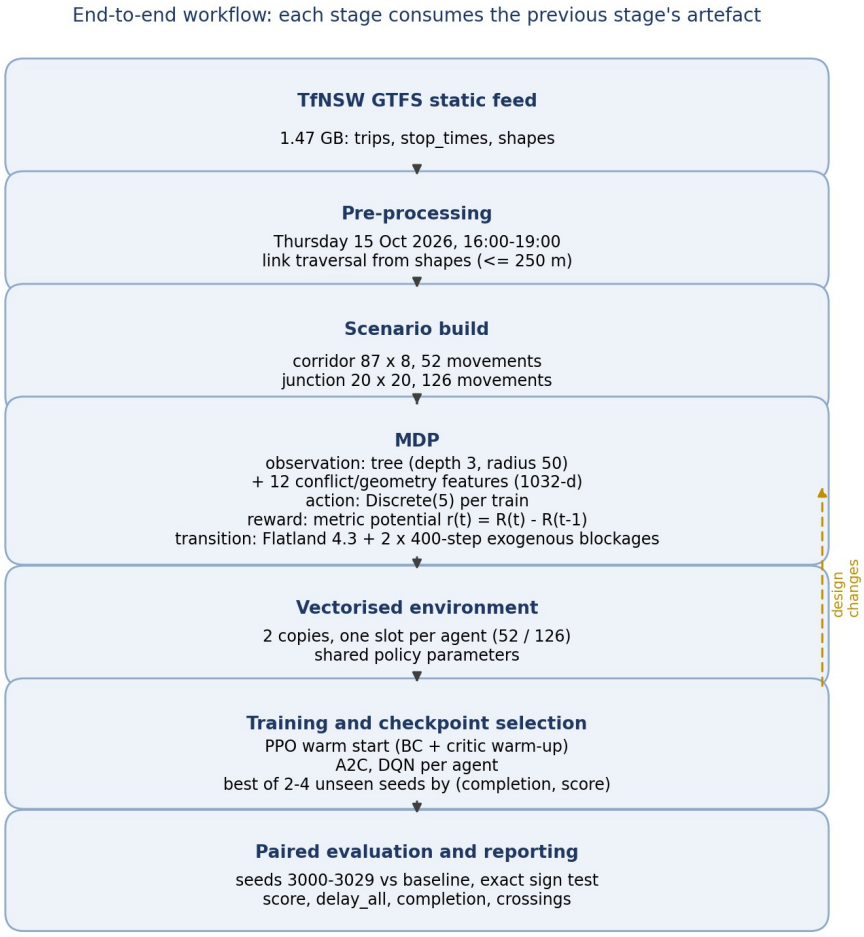


Figure 1: End-to-end workflow. Each stage consumes the artefact produced by the previous one; the dashed arrow marks the design loop that re-ran the evaluation after every change to observation, reward or regime.

- Service extraction: the GTFS static feed (about 1.4 GB) is filtered to the Thursday 15 October 2026 service between 16:00 and 19:00, with link traversal times derived from the shapes.
- Scenario construction: two Flatland levels and two timetables, the corridor (87 x 8, 52 movements) and the junction (20 x 20, 126 movements).
- MDP construction: level, timetable and disruption regime, with a tree observation, a per-train discrete action and the metric potential as reward.
- Vectorisation and training: two copies of the fleet for PPO, one for A2C and DQN, one sub-environment per train, shared policy parameters.
- Checkpoint selection: PPO is scored every 100k steps on unseen seeds 9001 to 9004 (thirteen evaluations in the shipped curve); A2C and DQN every 400k steps on two seeds and the checkpoint is chosen by completion first, then score.
- Paired evaluation and reporting: the corridor arm and its baseline on the same 30 held-out seeds (3000 to 3029), scored on the untouched metric, with an exact sign test.

2.2 System architecture

The notebook is both the deliverable and the assembly point. It installs the dependencies, defines the scenarios, the observation builder, the environment factory, the baseline, the training loop and the evaluator, and imports the shared modules in scripts/ instead of duplicating them, so the notebook, the training scripts and the reported tables cannot drift apart in the sense that matters: one environment definition, one observation, one regime, one evaluator for the reported pairs. Rendered frames are drawn with make_env while evaluation uses make_parallel_env, so a rendered score is indicative and every reported paired number comes from eval_pair; the internal scores in Table 3 come from the training scripts' own evaluator, and the Table 4 counts from crossing_audit.py. Twenty scripts ship (eighteen analysis, two report builders); Appendix A maps the modules that matter, and the notebook is the only entry point outside scripts/, calling them rather than re-implementing them.

The learning layer uses one shared multi-layer perceptron for all trains: 1032 inputs, two hidden layers (256 units for PPO, 128 for A2C and DQN), and a five-way action head. The observation is a flattened, normalised tree of depth 3 within a radius of 50 cells, plus twelve conflict and geometry features: slack and its rank, distance to the first occupied cell, distance to oncoming traffic, malfunctioning trains on my route, my own malfunction countdown, current speed, whether I am on the map, remaining route length, queue depth, distance to the nearest blocked cell ahead, and distance to my next commit crossover. The last two exist because a zeroed rail cell is invisible to occupancy features, so without them the crossing decision is not expressible.

All reported arms were trained with two shaping bonuses on top of the metric potential rather than with the bare metric: +50 for every train that arrives and -1 for every step a train is stuck, charged at the same 1/100 scale as the potential (runs/xb_b/config.json, ShapedMetricRewards). The disclosure matters twice over: the reported arm is not the bare-metric arm, and the arrival bonus is what separates a train that completes from one that is merely late.

Four design experiments fixed this architecture before the reported matrix was run. First, the reward: paid only at the end of an episode, a terminal charge of about 1000 arrives discounted to 0.37 of its size at the shipped gamma = 0.999, so the metric is paid as a potential instead, with the identity verified per agent and per term. Second, the observation horizon: at the default radius of 2 the distance features are effectively binary; at 50 they keep ordinal distance, which is what lets a train see a blockage far enough ahead to act. Third, the disruption layer: a train-borne failure that fires on arrival is suppressed

by the very behaviour being measured. Fourth, the warm start: from a random initialisation none of the three algorithms finds the crossing, so the reported corridor arm is PPO fine-tuned from a behaviour-cloned detour baseline, and Section 3.3 shows what the two independent algorithms do instead.

2.3 Environment and interface

Flatland's own renderer is the interface. The assessment permits a renderer in place of a separate GUI, and using the simulator's renderer means every figure comes from the same process the agent is trained in, produced headlessly with the PIL backend so the figures are reproducible. It shows the rail grid with its switches and crossovers, one coloured marker per train, and a red X on every cell that may not be entered: an exogenous blockage on the corridor, a train standing in malfunction on the junction.

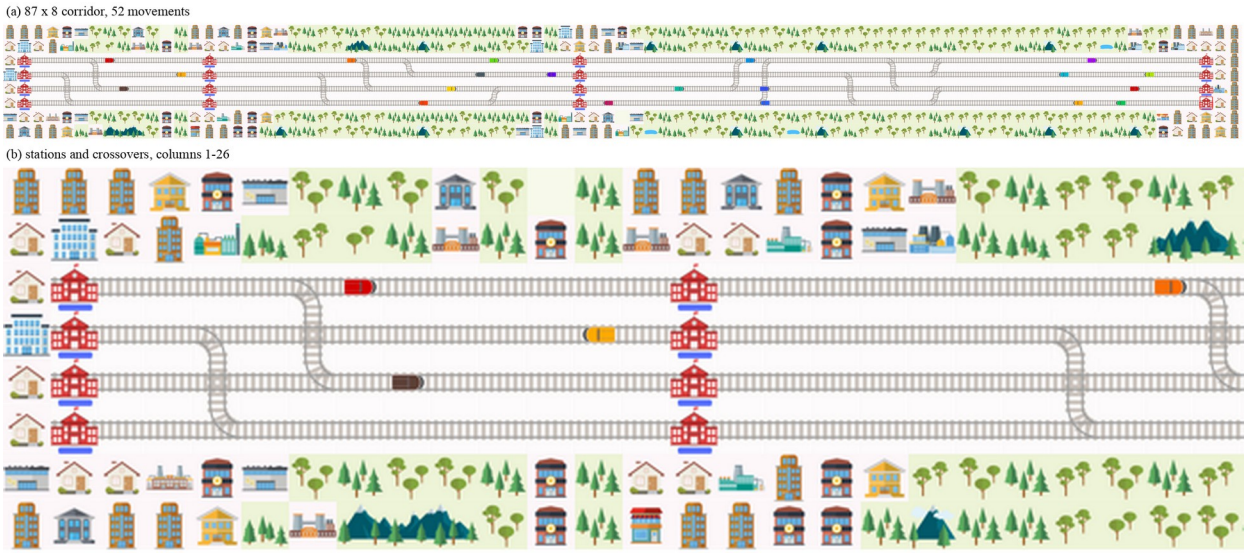


Figure 2: The corridor. (a) The full 87 x 8 level with 52 movements. (b) Columns 1 to 26 at native scale, showing Strathfield, Burwood and the switch ladders.

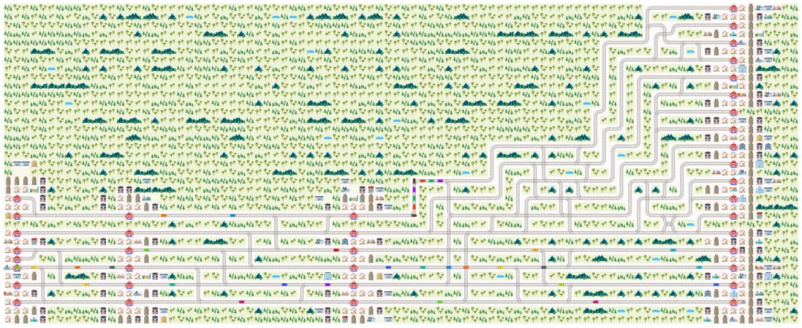


Figure 3: The junction. A 20 x 20 level where the Suburban pair and the Old Main South converge and diverge, carrying 126 scheduled movements.

2.4 The disruption regime

This section is the methodological core, because the first two versions of the regime were wrong in ways that inflated or erased the effect being measured.

Why a train-borne failure is not neutral. Firing a failure when a train reaches a cell makes the disruption a function of the policy's own trajectory, with two failure modes. A policy that reroutes early can prevent the incident from firing at all: in our own testing a rerouting checkpoint fired none of the incidents it was given and scored exactly as it did with no disruption, which is recorded in

results/all_arms.md as a log observation because that checkpoint is not shipped. And placing the failure anywhere between a station's escape crossover and its platform usually puts it where every follower has already passed the crossover: in the retired band regime none of the 9 sampled incidents left a train on the approach side, so there was no decision to make (results/all_arms.md).

The exogenous design. On the corridor a blockage removes a cell's rail transitions outright at a seed-drawn step and restores them after 400 steps, so it exists whether or not a train goes near it. Two are drawn per episode, one per direction, from the cells just past each commit crossover; the pool spans both up rows (2 and 4) and the down-local row (5), because the down-suburban row carries no commit crossover. A blockage on up_local just past Central catches a train that changed track at Burwood to dodge a blockage on up_sub. Activation is deferred while a cell is occupied, because removing the transitions under a standing train would strand it permanently; if the cell stays occupied for 60 steps the incident is dropped rather than fired. Both arms are therefore given the same cells at the same seed-drawn steps; because activation waits for an occupied cell to clear (at most 60 steps), the fired step can shift slightly between arms, and the pairing is by seed and blockage plan rather than by step index.

Duration is the lever that makes crossing pay. The detour between the two up tracks costs two cells, so a blockage shorter than that is not worth avoiding. The shipped regime holds a cell for 400 steps, 40 per cent of the 999-step budget and half of the 796-step episode the corridor actually runs, which is the regime the reported arm was trained and evaluated on.

The junction keeps a train-borne regime. It has no pair of parallel tracks, so an exogenous blockage would have no alternative route to measure. It uses three 220-step failures inside the running line, never on a station cell, with durations fixed so that every episode is late.

2.5 Where it runs

The regime and the policy are CPU-bound. Flatland steps the fleet in Python and the 1032-d tree observation is rebuilt for every agent at every step, so the network is not the bottleneck. The shipped corridor run took 1,865 s for 1,224,704 timesteps including thirteen four-episode evaluations (runs/xb_b/eval.csv), about 660 environment steps per second, and once startup is paid each evaluation interval costs about 148 s (nine of eleven between 143 and 155 s, the remaining two 118 s and 181 s), so the evaluator rather than the optimiser sets the wall clock. We could not run this PettingZoo stack under SubprocVecEnv at all (it hangs at reset in our runs), so every timing here is single-process. The shipped corridor run was trained on an x86_64 workstation with an RTX 3080 (system_info recorded in runs/xb_b/best_model.zip: Linux 7.2.10, Python 3.11.17, torch 2.14.1+cu130, SB3 2.9.0); the same stack also runs unchanged on the aarch64 Oracle relay (2 vCPU, 11 GB) used for development in this project; the twelve tests pass on both (results/test_run.txt).

3 Results and evaluation

This section reports the disrupted condition - the block regime on the corridor, train-borne failures on the junction - which is the only condition with headroom. With no disruption the corridor baseline is on time and scores exactly 0.0000, and so does the trained arm (results/clean_normal.csv), so a normal-condition comparison has nothing to explain.

3.1 Experimental settings

Item	Corridor	Junction
Scenario	network-simple-compact	granville-harris-park
Level	87 x 8, four through tracks	20 x 20 Y-link junction
Movements	52	126
Episode budget	999 steps	757 steps
Disruption	2 exogenous blockages, 400 steps	3 train-borne failures, 220 steps
Observation	tree depth 3, radius 50, plus 12 features (1032)	same
Action	Discrete(5) per train	same
Held-out seeds	3000 to 3029	3000 to 3029
Checkpoint selection	unseen seeds, completion then score	same
Baseline	forward-only reflex	scripted route follower

Table 1: Experimental settings. The maps share the observation, action space and evaluation protocol; they differ in disruption mechanism because only the corridor has a switchable pair of parallel tracks.

The score is the assessment metric normalised to [-1, 0]. Each train's cost is capped at one episode, summed over all agents and divided by the number of agents and the episode budget. A second statistic, delay_all, charges every agent its arrival minus its scheduled arrival, using the episode end for a train that never arrives, so a collapsed policy cannot look punctual by dropping unfinished trains from a survivors-only mean.

The corridor arm is evaluated with deterministic actions on seeds 3000 to 3029; A2C and DQN are reported from their selection seeds only, as Section 3.6 says. Because the baseline and the arm run on the same seed, the same env and the same blockages, the comparison is paired within seed: a delta is computed per seed and the report gives its mean, median, win count and an exact two-sided sign test. Training and checkpoint selection never see these seeds.

3.2 Pre-processing

- Service extraction: one Thursday service period, with rail links rather than straight lines giving traversal times.
- Corridor level: four adjacent through tracks with the ladder rows removed; the compact map leaves a two-cell diversion premium at 7 of the 15 stop types (results/station_access_compact.json).
- Junction level: a 20 x 20 level from the published track plan, with the turn sequence read out of the timetable as a route dictionary.
- Deadlines and episode budget: calibrated from a reference rollout, with the budget extended past the last arrival because the serialised value cut 15 junction trains off mid-journey.
- Multi-cell targets: the target set is rebuilt from the agent's own last waypoint group after every reset, otherwise a diverted train can never finish.
- Dense reward: the metric as a potential, verified against DefaultRewards per agent and per term.
- Metric hygiene: delay_all replaced a total-delay statistic that went NaN when nothing finished and negative when few did.
- Regime hygiene: the blockage is exogenous, so the delayed condition cannot collapse into the normal one.

3.3 Corridor: only the warm-started PPO reroutes

Arm	Baseline	Arm	Delta	Wins	Sign test p	Completion	delay_all
PPO	-0.0662	-0.0005	+0.0657	30/30	< 1e-4	1.000 -> 1.000	3321 -> -70

Table 2: Corridor, 30 paired held-out seeds under the block regime. Only PPO has the full paired test; Table 3 reports what the other two algorithms did in training.

PPO improves the score from -0.0662 to -0.0005, a mean paired gain of +0.0657 with a median of +0.0648. It wins all 30 seeds and every train completes in both arms, so the gain is punctuality rather than cancellation: total lateness falls from 3321 steps to effectively zero. The on-time share moves only from 0.625 to 0.635: the gain is in how late the affected trains are, not in which trains complete.

Under the clean condition the same checkpoint is inert. On three seeds it scores 0.0000, exactly as the forward-only baseline does (results/clean_normal.csv), and the decision audit records 546 visits to a commit crossover with a clear track and not one turn (results/clean_audit.csv). With the blockage suppressed but the Poisson malfunction layer left on, the arm is indistinguishable from the baseline: it scores -0.0133 against -0.0165, a paired +0.0032 on three seeds, 3 of 3 but p = 0.25 (results/clean_delayed_none.csv). The crossings in Table 4 therefore belong to the blockage, not to the policy's habit.

Arm	Best checkpoint	Internal score	Completion	Behaviour in Figure 4
PPO (warm start)	745k	-0.0005	1.000	crosses only when blocked
A2C	400k	-0.0563	1.000	no crossing in the panel
DQN (gamma 0.999)	400k	-0.3234	0.529	parks half the fleet

Table 3: Corridor training outcome per arm. Score and completion are the selection-seed numbers (9001 to 9004 for PPO, 9001 and 9002 for A2C and DQN), not the 30-seed paired numbers in Table 2. The behaviour column is a reading of the panels in Figure 4, which are rendered with make_env rather than the evaluator's make_parallel_env; the panel labels are the renderer's own episode scores and are quoted only there, not in this table.

The other two algorithms are the control for the claim that learning the crossing is not automatic. A2C oscillates between a near-baseline score and a parked fleet (-0.0563 at 400k, -0.5746 with completion 0.087 at 800k, -0.0736 at 1.2M) and, in the rendered episode, its best checkpoint never turns at a crossover. DQN with gamma = 0.999 collapses to parking by 800k and stays there, finishing with completion 0.000; its best checkpoint is 400k, where it still strands half the fleet. Neither arm was given the warm start, so the comparison isolates the recipe as well as the algorithm: on this map the reward alone does not pull a from-scratch learner to the crossing.

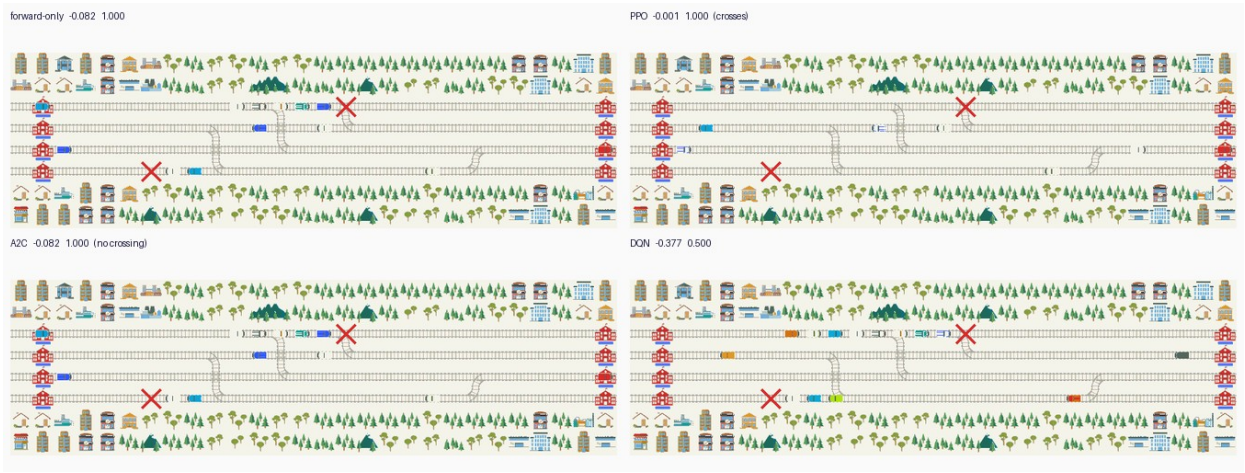


Figure 4: The four corridor arms on the same episode (seed 3001; step 232 for forward-only, PPO and A2C, step 300 for DQN where its queue has formed), cropped to the stretch where the two blockages land. The red X is a cell whose rail transitions were removed. Forward-only, A2C and DQN leave a queue behind the upper X; PPO's track ahead of it is clear because it changed at the previous crossover.

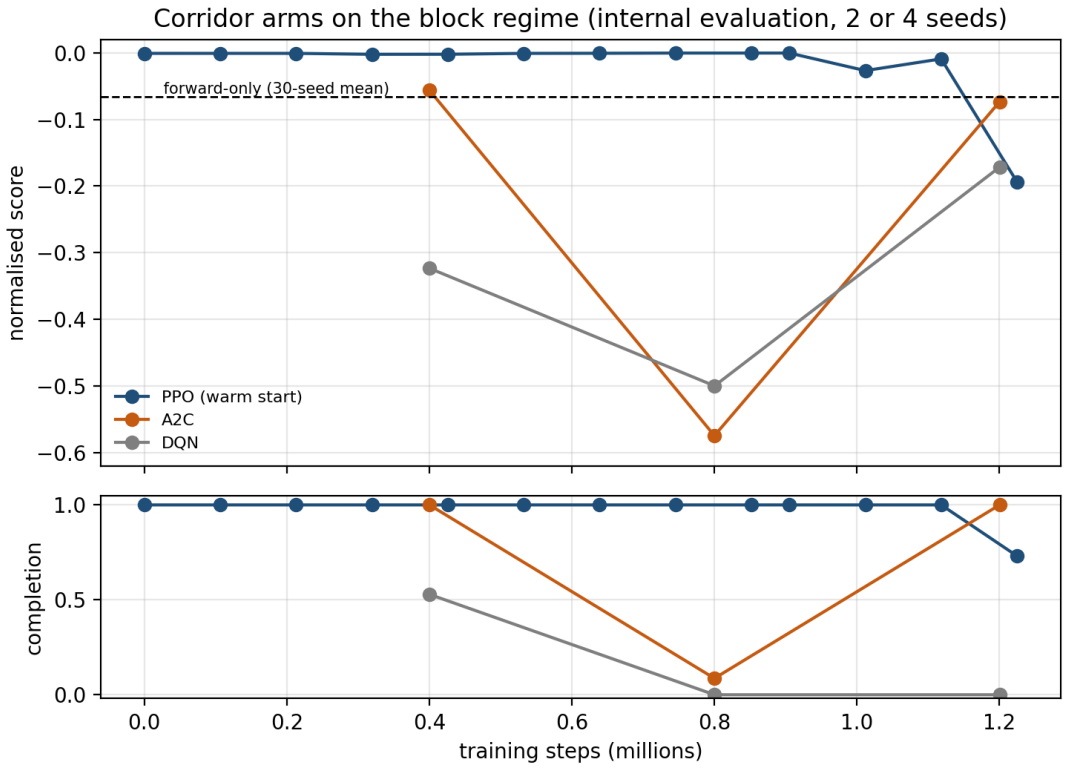


Figure 5: The three corridor arms during training, from the internal evaluation used for checkpoint selection. PPO starts from the behaviour-cloned checkpoint and stays near zero until it drifts at the end of the budget; A2C and DQN fall into the parking basin and only partly recover. The dashed line is the forward-only 30-seed mean.

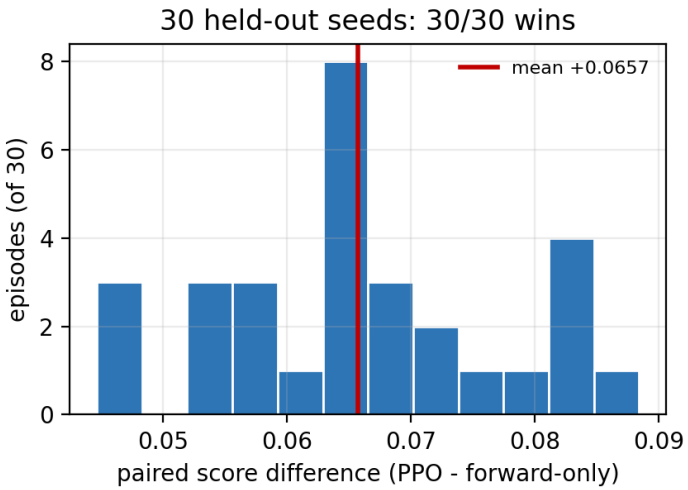


Figure 6: The 30 per-seed differences behind Table 2. All 30 are positive, with a mean of +0.0657 and a median of +0.0648.

3.4 The crossings are conditional

A score gain alone does not prove that the policy changes track for the right reason, since a policy that crosses at every crossover also gains on this map: both same-direction tracks accept the same platforms. The audit counts the decisions themselves. For every agent-step standing on a commit crossover it records whether a blocked cell lay ahead on that train's own row within 60 cells ahead, and whether the agent chose a lateral action. The per-crossover table is shipped as results/crossing_audit_block.csv by scripts/crossing_audit.py.

Crossover	Track	Blocked visits	Turns	Open visits	Turns
(2,6)	up_suburban	22	22	30	0
(2,25)	up_suburban	2	2	28	0
(2,62)	up_suburban	0	0	46	0
(4,51)	up_local	18	18	58	0
(5,4)	down_local	0	0	14	0
(5,22)	down_local	19	19	14	0
(5,59)	down_local	40	20	12	0
Total		**101**	**81**	**202**	**0**

Table 4: Per-crossover decision audit over seeds 3000 and 3001, shipped as results/crossing_audit_block.csv. Blocked means an impassable cell ahead on the train's own row within 60 cells. (5,59) is the only crossover with blocked visits that did not always turn (20 of 40); the CSV records counts rather than per-visit causes, so none is claimed.

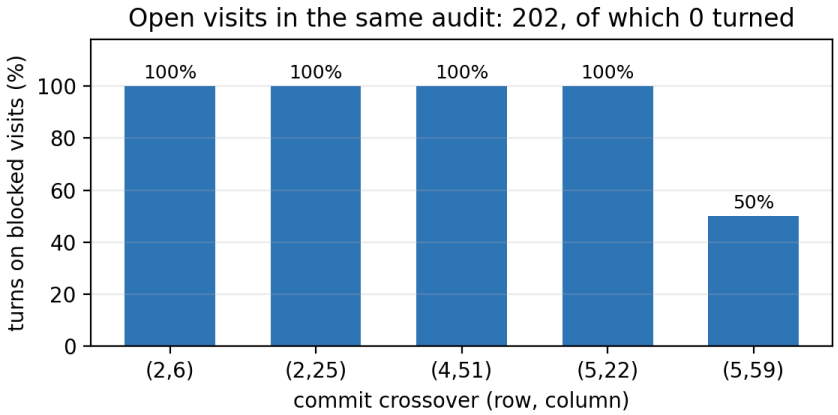


Figure 7: Turn rate on the visits where the track ahead was blocked. Only the five crossovers with a blocked visit in this sample are plotted.

Of 101 agent-steps standing on a crossover with a blocked cell ahead, 81 produced a turn; of 202 with a clear track, none did. The audit records one decision per visit to a commit cell, so 81 turns over two seeds is about 40 diverts per episode, and the rate is not uniform: four crossovers turn on every blocked visit and (5,59) on 20 of 40. The behaviour is therefore state-dependent, not a blanket lane swap, and the A2C and DQN animations in Figure 4 show what the blanket alternative looks like.

3.5 The junction: a loss, and a changed failure mode

The junction is reported as a loss. Services there have no intermediate stops, so each movement is a single origin-to-target run and the network is a pure arbitration problem: 126 trains share a small set of junction cells in a fixed order, which the scripted route follower reads out of the timetable. A learner sees only a local tree observation.

Arm	Baseline	Arm	Delta	Wins	Completion (base -> arm)
PPO (30 seeds, 1030-d)	-0.1769	-0.0251	+0.1518	30/30	1.000 -> 0.000
A2C (30 seeds, 1030-d)	-0.1769	-0.0251	+0.1518	30/30	1.000 -> 0.000
DQN (30 seeds, 1030-d)	-0.1769	-0.0208	+0.1561	30/30	1.000 -> 0.183
PPO (current, 2 seeds)	-0.1823	-0.2400	-0.0577	0/2	1.000 -> 0.468

Table 5: Junction results. The first three rows are the recorded 30-seed runs from before the two observation features were added, and they are kept as measured: that policy parked the fleet. The last row is the current checkpoint selected by completion, which departs and jams; it is a two-seed evaluation shipped as results/junction_current.csv, whose arm score is identical on both seeds (its episode is seed-invariant, so treat it as one behaviour). The 30-seed rows are seed-invariant in the same way - one distinct arm score across 30 different baselines - so their 30 of 30 means one parked or jammed behaviour beating 30 baselines, not 30 independent trials. Those rows come from results/v2_*.csv; the two are not directly comparable and are labelled accordingly.

The old rows are the metric pathology in its purest form: every learned arm scores about +0.15 better than the route follower while completing 0 to 18 per cent of the trains, because a cancellation costs about 19 cost units on this map while a dispatch into the jam is charged its full delay. The current checkpoint shows the other side of the same pathology. Selected by completion rather than score, it sends every dispatched train into the throat and jams: 59 of 126 complete and the other 67 are stranded, scoring -0.2400, which is worse than the scripted baseline at -0.1823 with 126 of 126 home. Neither outcome is a working junction policy: one is gaming the metric, the other is losing to the schedule. A global controller, or imitation of the route follower, is what the map needs.

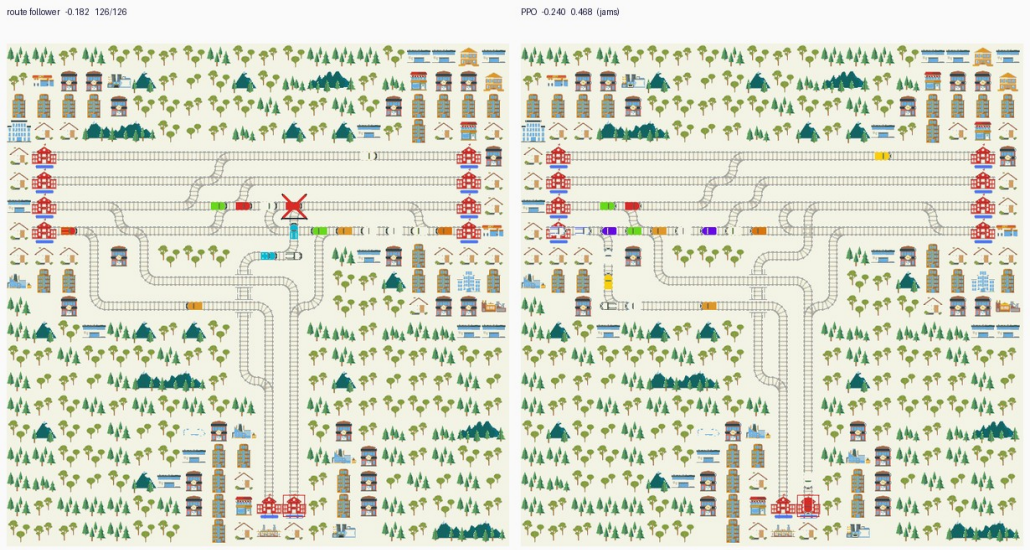


Figure 8: The junction at step 156 under the train-borne regime, rendered with make_env (so the scores shown are indicative; the paired numbers are in Table 5). Left: the scripted route follower, with the held train marked and the queue cleared around it, 126 of 126 trains home. Right: the selected PPO checkpoint, which dispatches into the throat and jams, 59 of 126 home.

3.6 Limitations

- Only PPO has the full 30-seed paired test on the corridor. A2C and DQN are reported from their selection seeds and from the rendered episode; a paired run for them would strengthen the three-algorithm claim and is the obvious next experiment.
- The corridor decision is local: one crossover per blocked stretch and a two-cell detour. The result should not be read as evidence that per-agent RL can arbitrate a junction.
- The disruption volume is a modelling choice, not data: 2 x 400 steps keeps completion at 1.000 while making every affected trip late.
- From-scratch learning does not find the crossing, and the A2C and DQN arms demonstrate it independently. The reported corridor arm is PPO warm-started from a behaviour-cloned detour baseline with the critic warmed up first: imitation-then-RL, not pure exploration.
- The warm start inherits the teacher's rule, so the policy cannot beat the teacher's own score of -0.0005 (results/all_arms.md). The claim is that it reaches that local optimum, not that it exceeds it.
- The junction table mixes two observation versions: the 30-seed rows predate the two feature additions, and their checkpoints no longer load. They are reported as measured and labelled, not silently refreshed.
- Evaluation is entirely simulated. The timetable and the track layout are real; there is no operational validation and no human factors layer.
- The 30 held-out seeds vary the blockage placement only: fleet, timetable and deterministic actions are fixed, and a single training seed per arm was run. The mean delta is reported with a sign test but without a confidence interval, and no corridor arm is compared against the stronger scripted distance-map baseline.

4 Conclusion

A local policy can learn to reroute, but only if the evaluation makes the decision real, and only with a recipe that can find a sparse action. On a four-track corridor cut by 400-step exogenous blockages, PPO changes track at the commit crossover just before the blockage and turns a paired deficit of -0.0662

into -0.0005 over 30 held-out seeds, winning every seed without losing a train. The same checkpoint does nothing when nothing is blocked, and an audit of 303 crossover visits shows that its crossings follow the blockage rather than the map. The two independent algorithms do not get there: A2C returns to the forward reflex and DQN parks half the fleet, which is what makes the corridor result a statement about the recipe as much as about the algorithm.

The result depended on fixing the measurement first. A train-borne failure fires when a train arrives, so a policy that reroutes suppresses the incident, and a failure placed deep inside a station's escape band leaves every follower already past the crossover. Removing the cell's transitions instead makes the blockage exogenous, keeps both arms on the same incidents, and turns the delayed condition into a genuine decision. The junction remains a loss, and its two failure modes bracket the metric's pathology: parking scores better than attempting, and attempting scores worse than the schedule.

Appendix A Module map

Module	Responsibility
nb_harness.py	Executes the notebook's cells so the CLI tools use exactly the notebook's code
regimes.py	Map geometry (seven commit crossovers) and the per-scenario regime selector
disruptions.py	Both disruption mechanisms: train-borne failures and the exogenous blockage
priority_obs.py	The twelve conflict/geometry observation features (1032-d input)
metric_rewards.py	The metric as a dense potential, plus the shaped variant
train_ppo.py	PPO training, dense reward, checkpoint selection, critic warm-up
train_algo.py	A2C and DQN on the same environment and evaluator
eval_pair.py	Paired evaluation of an arm against its baseline on held-out seeds
bc_warmstart.py	Detour teacher, behaviour-cloning labels, warm-start checkpoint
crossing_audit.py	Score, per-crossover decisions, and regime exposure (liveness)
diag_action_collapse.py	Action-distribution check for a collapsed policy
render_policy_gif.py	Episode figures, red-X overlay and optional diagnostic markers
verify_rewards.py	Proves the dense reward reproduces the metric per agent and per term
s2_env.py, s2_assembly.py	Junction environment assembly (rail, fleet, timetable)
build_report.py	This document; numbers are copied by hand from the shipped CSVs and figures

Table A1: The modules this report depends on, fifteen analysis modules plus this document's builder, out of twenty shipped scripts (eighteen analysis, two report builders). The notebook imports five directly; the rest are command-line entry points. The notebook, not the scripts, is the graded artefact.

Appendix B Reproducing the reported results

All commands run from the repository root. Steps 1 and 2 rebuild the corridor checkpoint; the evaluation and audit steps then use it. runs/ is not tracked, so on a fresh clone either run steps 1 and 2 first or point --model at your own checkpoint.

```
python scripts/bc_warmstart.py collect --episodes 20 --out runs/bc_block/data.npz

python scripts/bc_warmstart.py train --data runs/bc_block/data.npz --out runs/bc_block/bc3.zip --epochs 8 --turn-weight 20

RL_A3_REGIME=block python scripts/train_ppo.py --scenario network-simple-compact --disruption few_long --incidents 2 --shaping --priority --observation-radius 50 --arrival-bonus 50 --stuck-penalty 1 --timesteps 1200000 --lr 0.0001 --freeze-policy-steps 200000 --eval-every 100000 --pretrain runs/bc_block/bc3.zip --out runs/xb_b

RL_A3_REGIME=block python scripts/eval_pair.py --scenario network-simple-compact --baseline forward --disruption few_long --incidents 2 --priority --observation-radius 50 --model runs/xb_b/best_model.zip --seeds "$(seq -s, 3000 3029)" --out results/pair_block_ppo.csv

RL_A3_REGIME=block python scripts/crossing_audit.py score --model runs/xb_b/best_model.zip --seeds 3000,3001,3002 --incidents 2
```

```
RL_A3_REGIME=block python scripts/crossing_audit.py decisions --model runs/xb_b/best_model.zip --seeds 3000,3001 --incidents 2 --out results/crossing_audit_block.csv

RL_A3_REGIME=block python scripts/train_algo.py --algo a2c --incidents 2 --eval-every 400000 --eval-seeds 9001,9002 --out runs/a2c_block

RL_A3_REGIME=block python scripts/train_algo.py --algo dqn --incidents 2 --gamma 0.999 --eval-every 400000 --eval-seeds 9001,9002 --out runs/dqn_block

python scripts/render_policy_gif.py --markers block --model runs/xb_b/best_model.zip --scenario network-simple-compact --seed 3001 --incidents 2 --stride 1 --width 4200 --out results/figures/corridor_ppo_block.gif

python scripts/eval_pair.py --scenario network-simple-compact --conditions normal --baseline forward --priority --observation-radius 50 --model runs/xb_b/best_model.zip --seeds 3000,3001,3002 --out results/clean_normal.csv

python scripts/crossing_audit.py decisions --variant none --model runs/xb_b/best_model.zip --seeds 3000,3001,3002 --out results/clean_audit.csv

python scripts/eval_pair.py --scenario network-simple-compact --conditions delayed --baseline forward --priority --observation-radius 50 --model runs/xb_b/best_model.zip --seeds 3000,3001,3002 --out results/clean_delayed_none.csv

python scripts/make_report_figures.py && python scripts/build_report.py

python -m unittest discover -s tests && python scripts/verify_rewards.py --policy forward --seed 2000 --incidents 2
```

Portability. The reported training run and the clean-control evaluations ran on an x86_64 workstation with an RTX 3080 (system_info in runs/xb_b/best_model.zip, and the RTX box named in requirements.txt); the notebook, the figures and this report also run on an aarch64 Oracle relay (2 vCPU, 11 GB) with the same pinned stack; the twelve tests pass on both (results/test_run.txt). The environment is CPU-bound, so the relay is a valid host even without a GPU, and its 2 vCPU is the practical limit on running arms concurrently.